AlphaZero: fundamentos, formulación matemática, búsqueda, aprendizaje y construcción de estrategia

Nota de trabajo técnica

Dr. Pedro E. Colla

29 de septiembre de 2026

Índice

1. Motivación general **2**

2. Marco matemático: juegos como procesos de decisión **2**
2.1. Estados, acciones y transición . . . 2
2.2. Recompensa terminal . . . 3
2.3. Política . . . 3
2.4. Función de valor . . . 3

3. El problema de "la mejor estrategia" **4**

4. La red neuronal de política y valor **4**
4.1. Función conjunta . . . 4
4.2. Cabeza de política . . . 5
4.3. Cabeza de valor . . . 5
4.4. Representación del tablero . . . 5

5. Arquitectura residual **5**

6. Monte Carlo Tree Search **6**
6.1. Idea básica . . . 6
6.2. Las cuatro fases conceptuales . . . 6

7. Selección mediante PUCT **6**
7.1. Regla básica . . . 6
7.2. Interpretación . . . 7
7.3. Efecto de c_{puct} . . . 7

8. Expansión de un nodo **7**

9. Retropropagación **7**

10. De las visitas del árbol a una política mejorada **8**
10.1. Temperatura alta . . . 8
10.2. Temperatura baja . . . 8

11. Ruido de Dirichlet **9**

12. El ciclo de autojuego **9**

13.El objetivo de aprendizaje **9**
13.1. Pérdida de valor . . . 10
13.2. Pérdida de política . . . 10
13.3. Regularización . . . 10

14.Actualización por descenso de gradiente **10**

15.Por qué la búsqueda puede ser mejor que la red **10**

16.AlphaZero como iteración de políticas generalizada **11**

17.Relación con Bellman **11**

18.Valor de una acción **12**

19.Cómo se define una jugada “mejor” **12**

20.Política de entrenamiento versus política de juego **12**

21.Aprendizaje desde cero **13**

22.Bootstrap y problema inicial **13**

23.Replay buffer **13**

24.Distribución cambiante de datos **14**

25.La interacción búsqueda–red **14**

26.Por qué el valor terminal puede afectar decisiones lejanas **14**

27.Diferencia entre retropropagación MCTS y backpropagation neuronal **15**
27.1. Backup en MCTS . . . 15
27.2. Backpropagation neuronal . . . 15

28.No existe una garantía práctica de optimalidad **15**

29.Error de aproximación **16**

30.Presupuesto de búsqueda **16**

31.Uso de batching **16**

32.Virtual loss **16**

33.Entrenamiento síncrono y asíncrono **17**

34.Distribución de visitas como mejora de política **17**

35.Interpretación como búsqueda amortizada **17**

36.Relación entre política y valor **18**

37.Aperturas **18**

38.Finales **18**

39.Estados transpuestos **19**

40.Simetrías **19**

41.Calibración del valor **19**

42.Política multimodal **19**

43.Crédito temporal **20**

44.Una formulación compacta del algoritmo **20**

45.Pseudocódigo conceptual **21**

46.Qué significa que AlphaZero "aprende estrategia" **21**

47.Conocimiento implícito **22**

48.Auditabilidad **22**

49.Diagnóstico de aprendizaje **22**

50.Sobreajuste al autojuego **23**

51.Evaluación externa **23**

52.Catastrophic forgetting **23**

53.Parámetros esenciales **23**

54.Efecto del número de simulaciones **24**

55.Profundidad versus anchura **24**

56.Comparación con minimax **24**

57.Comparación con Q-learning **24**

58.Comparación con policy gradient puro **25**

59.El papel de MCTS como "teacher" **25**

60.Riesgo de auto-confirmación **25**

61.Convergencia: qué puede y qué no puede afirmarse **26**

62.Una lectura sistémica **26**

63.Implicancias para implementaciones experimentales **26**

64.Qué debería medirse **27**

65.Desacople entre generación y aprendizaje **27**

66.Edad de política y coherencia **28**

67.Conclusiones **28**

68.Resumen matemático final **29**

A. Glosario **29**

B. Referencias conceptuales sugeridas **30**

Resumen

AlphaZero es un algoritmo de aprendizaje por refuerzo profundo que aprende a jugar juegos de información perfecta y suma cero a partir de autojuego, sin utilizar partidas humanas como fuente necesaria de supervisión. Su arquitectura combina una red neuronal que estima simultáneamente una política y un valor, con una búsqueda Monte Carlo Tree Search (MCTS) guiada mediante una regla de exploración tipo PUCT. La búsqueda produce una política mejorada respecto de la política cruda de la red; el autojuego genera datos; y esos datos se utilizan para entrenar una nueva red. El proceso se repite iterativamente.

Esta nota desarrolla de forma autocontenida los conceptos matemáticos y algorítmicos necesarios para comprender el funcionamiento de AlphaZero, incluyendo la formulación como proceso de decisión de Markov, la representación del estado, la red de política-valor, la búsqueda PUCT, el autojuego, la función de pérdida, la actualización de parámetros, la relación con iteración de políticas generalizada y las limitaciones teóricas del método. El objetivo es distinguir con precisión entre lo que AlphaZero calcula, lo que aproxima y lo que puede inferirse de sus resultados.

1. Motivación general

AlphaZero pertenece a la familia de algoritmos que combinan aprendizaje por refuerzo, redes neuronales profundas y búsqueda. Su idea central puede resumirse como un ciclo de mejora:

1. una red neuronal propone qué jugadas parecen prometedoras y estima el valor de una posición;
2. una búsqueda de árbol utiliza esas estimaciones para analizar selectivamente variantes;
3. la distribución de visitas del árbol se interpreta como una política mejorada;
4. se juega utilizando esa política;
5. el resultado final de la partida se propaga como señal de aprendizaje;
6. la red se entrena para aproximar las políticas mejoradas y los resultados observados;
7. el procedimiento se repite.

Esto crea una interacción entre dos mecanismos distintos:

- **generalización**: la red neuronal intenta capturar regularidades globales;
- **razonamiento local**: MCTS examina con mayor detalle la posición concreta.

La red sola es rápida pero imperfecta. La búsqueda sola sería muy costosa en espacios enormes. AlphaZero combina ambas.

2. Marco matemático: juegos como procesos de decisión

2.1. Estados, acciones y transición

Sea un juego determinista de dos jugadores, de información perfecta y suma cero. Denotemos:

$$s \in \mathcal{S}$$

al estado del juego, y

$$a \in \mathcal{A}(s)$$

a una acción legal en el estado s.
La transición determinista se escribe:

$$s' = T(s, a).$$

En ajedrez, por ejemplo, s contiene como mínimo:

- disposición de piezas;
- jugador al turno;
- derechos de enroque;
- posibilidad de captura al paso;
- información relevante para reglas de tablas.

2.2. Recompensa terminal

En la formulación original de AlphaZero, la señal principal de recompensa aparece al final de la partida. Si z representa el resultado desde la perspectiva del jugador correspondiente, típicamente:

$$z = \begin{cases} +1, & \text{victoria}, \\ 0, & \text{tablas}, \\ -1, & \text{derrota}. \end{cases}$$

El juego es de suma cero, de modo que si para un jugador el resultado es $+1$, para el otro es -1.

2.3. Política

Una política es una distribución de probabilidad sobre acciones legales:

$$\pi(a \mid s).$$

Satisface:

$$\pi(a \mid s) \geq 0, \qquad \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) = 1.$$

La política puede verse como una regla probabilística de decisión.

2.4. Función de valor

El valor de un estado bajo una política π es:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[z \mid s].$$

En un juego de suma cero, un valor positivo indica ventaja esperada para el jugador cuya perspectiva se está utilizando.
Una política óptima, idealmente, satisface:

$$V^{*}(s) = \max_{\pi} V^{\pi}(s),$$

pero AlphaZero no calcula exactamente esta cantidad en juegos grandes. La aproxima mediante red neuronal y búsqueda.

3. El problema de “la mejor estrategia”

Es importante distinguir tres objetos diferentes:

1. la política cruda de la red neuronal;
2. la política refinada por MCTS;
3. la política verdaderamente óptima del juego.

La red produce:

$$p_\theta(a \mid s),$$

donde θ son los parámetros de la red.
La búsqueda produce otra distribución:

$$\pi_{\text{MCTS}}(a \mid s).$$

La política efectiva de AlphaZero durante el autojuego se deriva principalmente de las visitas del árbol, no simplemente del máximo de p_θ.
La política óptima real:

$$\pi^*(a \mid s)$$

es, en juegos enormes, desconocida.
Por tanto:

$$p_\theta \neq \pi_{\text{MCTS}} \neq \pi^*$$

en general.
AlphaZero intenta que, a medida que el entrenamiento progresa,

$$p_\theta \approx \pi_{\text{MCTS}},$$

y que el proceso conjunto acerque ambas políticas a decisiones de alta calidad.

4. La red neuronal de política y valor

4.1. Función conjunta

La red neuronal implementa una función:

$$f_\theta(s) = (\boldsymbol{p}, v),$$

donde:

$$\boldsymbol{p} = (p(a_1 \mid s), \ldots, p(a_n \mid s))$$

es una distribución sobre las acciones posibles y

$$v \in [-1, 1]$$

es la estimación del resultado esperado desde el estado s.
Esto implica que una sola red responde dos preguntas:

1. **Política:** £qué jugadas parecen prometedoras?
2. **Valor:** £qué tan favorable parece la posición?

4.2. Cabeza de política

Internamente la red produce logits l_a. La política se obtiene mediante softmax:

$$p(a \mid s) = \frac{e^{l_a}}{\sum_b e^{l_b}}.$$

La política no es todavía la decisión final. Funciona como *prior* para orientar la búsqueda.

4.3. Cabeza de valor

La cabeza de valor produce un escalar:

$$v_\theta(s) \approx \mathbb{E}[z \mid s].$$

Normalmente se restringe a:

$$v_\theta(s) \in [-1, 1],$$

por ejemplo mediante una activación tanh.

4.4. Representación del tablero

En ajedrez, el estado se codifica como tensores espaciales. Conceptualmente:

$$X(s) \in \mathbb{R}^{H \times W \times C}.$$

Para un tablero de ajedrez:

$$H = W = 8,$$

y C contiene planos que representan piezas, turno, historial y otra información.
La ventaja de esta representación es preservar la estructura espacial del tablero.

5. Arquitectura residual

AlphaZero utiliza una red profunda con bloques residuales. Si x_l es la entrada a un bloque, la forma conceptual es:

$$x_{l+1} = \sigma\left(x_l + F(x_l; W_l)\right).$$

La conexión residual:

$$x_l \to x_l + F(x_l)$$

facilita el entrenamiento de redes profundas.
El cuerpo compartido de la red extrae características; luego se divide en:

$$\text{trunk} \longrightarrow \begin{cases} \text{policy head}, \\ \text{value head}. \end{cases}$$

6. Monte Carlo Tree Search

6.1. Idea básica

MCTS construye un árbol de posiciones alcanzables. Cada arista (s, a) almacena estadísticas tales como:

$$N(s, a) : \text{número de visitas},$$

$$W(s, a) : \text{suma de valores acumulados},$$

$$Q(s, a) : \text{valor medio},$$

$$P(s, a) : \text{prior de la red}.$$

Con:

$$Q(s, a) = \frac{W(s, a)}{N(s, a)}$$

si $N(s, a) > 0$.

6.2. Las cuatro fases conceptuales

Una simulación MCTS contiene cuatro etapas:

1. selección;
2. expansión;
3. evaluación;
4. retropropagación.

Estas etapas se repiten muchas veces desde la misma posición raíz.

7. Selección mediante PUCT

7.1. Regla básica

AlphaZero utiliza una forma de Upper Confidence Bound guiada por una política previa. Una expresión típica es:

$$a_t = \arg\max_a \left[Q(s, a) + U(s, a) \right],$$

donde

$$U(s, a) = c_{\text{puct}}\, P(s, a) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}.$$

Por tanto:

$$a_t = \arg\max_a \left[Q(s, a) + c_{\text{puct}}\, P(s, a) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)} \right].$$

7.2. Interpretación

El término:

$$Q(s, a)$$

favorece jugadas que ya han producido buenos resultados.
El término:

$$U(s, a)$$

favorece jugadas:

- poco visitadas;
- consideradas prometedoras por la red;
- relevantes en relación con el número total de visitas del nodo.

Esta tensión se denomina:

$$\text{exploración} \quad \text{vs} \quad \text{explotación}.$$

7.3. Efecto de c_{puct}

Si c_{puct} es grande, la búsqueda explora más.
Si c_{puct} es pequeño, la búsqueda explota antes las variantes que ya parecen mejores.
No existe un valor universalmente óptimo; su elección depende de la escala de la red, el juego y el presupuesto de simulaciones.

8. Expansión de un nodo

Cuando la búsqueda alcanza por primera vez un estado s_L, la red se evalúa:

$$(\boldsymbol{p}, v) = f_\theta(s_L).$$

Para cada acción legal a:

$$P(s_L, a) = p_\theta(a \mid s_L).$$

El valor:

$$v_\theta(s_L)$$

se utiliza como estimación del resultado de la posición sin tener que jugar una simulación aleatoria hasta el final.
Ésta es una diferencia importante respecto del MCTS clásico basado en rollouts aleatorios.

9. Retropropagación

Supongamos que una simulación alcanza un estado hoja con evaluación:

$$v.$$

Ese valor se propaga por las aristas visitadas.
Para una arista:

$$N(s,a) \leftarrow N(s,a) + 1,$$

$$W(s,a) \leftarrow W(s,a) + v,$$

$$Q(s,a) \leftarrow \frac{W(s,a)}{N(s,a)}.$$

Debido a la alternancia de jugadores, debe corregirse la perspectiva del valor. Si un nodo se evalúa desde el jugador que mueve allí, al retroceder un nivel:

$$v \leftarrow -v.$$

Esta alternancia de signo es fundamental en juegos de suma cero.

10. De las visitas del árbol a una política mejorada

Después de M simulaciones, cada acción raíz posee un número de visitas:

$$N(s,a).$$

AlphaZero construye una política de búsqueda:

$$\pi(a \mid s) = \frac{N(s,a)^{1/\tau}}{\sum_b N(s,b)^{1/\tau}},$$

donde τ es un parámetro de temperatura.

10.1. Temperatura alta

Para τ relativamente grande, la distribución es más plana:

$$\pi(a \mid s) \approx \text{exploratoria}.$$

10.2. Temperatura baja

Cuando:

$$\tau \to 0,$$

la política se aproxima a seleccionar:

$$a^* = \arg\max_a N(s,a).$$

En entrenamiento suele utilizarse mayor exploración en las primeras jugadas y decisiones más deterministas después.

11. Ruido de Dirichlet

Para evitar que el autojuego se vuelva prematuramente determinista, AlphaZero perturba los priors en la raíz:

$$P'(s,a) = (1-\varepsilon)P(s,a) + \varepsilon\eta_a,$$

donde

$$\boldsymbol{\eta} \sim \mathrm{Dir}(\alpha).$$

Esto induce exploración estructurada.

El propósito no es mejorar directamente una jugada individual, sino incrementar la diversidad del conjunto de experiencias.

12. El ciclo de autojuego

Una partida de autojuego produce una secuencia:

$$s_0, a_0, s_1, a_1, \ldots, s_T.$$

Para cada estado s_t:

1. se ejecuta MCTS;
2. se obtiene π_t;
3. se elige una acción según π_t;
4. se avanza al siguiente estado.

Al finalizar la partida se obtiene:

$$z \in \{-1, 0, +1\}.$$

Cada posición se transforma en un ejemplo:

$$(s_t, \pi_t, z_t).$$

El conjunto de datos de entrenamiento es entonces:

$$\mathcal{D} = \{(s_i, \pi_i, z_i)\}_{i=1}^{N}.$$

13. El objetivo de aprendizaje

La red debe aprender simultáneamente:

1. a reproducir la política mejorada de MCTS;
2. a predecir el resultado final.

Una función de pérdida típica es:

$$\mathcal{L}(\theta) = (z - v_\theta(s))^2 - \boldsymbol{\pi}^\top \log \boldsymbol{p}_\theta(s) + \lambda\|\theta\|_2^2.$$

13.1. Pérdida de valor

$$\mathcal{L}_V = (z - v)^2.$$

Hace que:

$$v_\theta(s) \to z.$$

13.2. Pérdida de política

$$\mathcal{L}_P = -\sum_a \pi(a \mid s) \log p_\theta(a \mid s).$$

Es la entropía cruzada entre la política producida por MCTS y la política de la red.
Equivale a minimizar, salvo una constante:

$$D_{\mathrm{KL}}(\pi \parallel p_\theta).$$

13.3. Regularización

$$\mathcal{L}_R = \lambda \|\theta\|_2^2.$$

Su función es reducir el sobreajuste y limitar el crecimiento excesivo de parámetros.

14. Actualización por descenso de gradiente

Los parámetros se ajustan mediante:

$$\theta_{k+1} = \theta_k - \eta \nabla_\theta \mathcal{L}(\theta_k),$$

donde:

$$\eta$$

es la tasa de aprendizaje.
En la práctica se utilizan optimizadores por mini-batches y variantes de descenso estocástico.

15. Por qué la búsqueda puede ser mejor que la red

La red devuelve una evaluación local:

$$f_\theta(s) = (p, v).$$

Pero MCTS construye información adicional mediante múltiples expansiones.
En términos conceptuales:

$$p_\theta \xrightarrow{\text{MCTS}} \pi.$$

La búsqueda puede descubrir que una jugada inicialmente poco probable conduce a posiciones fuertes, o que una jugada aparentemente atractiva tiene respuestas adversas.
Por eso:

$$\pi$$

se considera un objetivo mejorado respecto de:

$$p_\theta.$$

16. AlphaZero como iteración de políticas generalizada

El procedimiento puede interpretarse como una forma aproximada de *Generalized Policy Iteration*.

Sea:

$$\pi_k$$

la política representada por la red en una iteración.

MCTS produce:

$$\pi'_k = \text{Search}(\pi_k, V_k).$$

Luego el entrenamiento ajusta:

$$\pi_{k+1} \approx \pi'_k.$$

Simultáneamente:

$$V_{k+1} \approx V^{\pi'_k}.$$

El ciclo conceptual es:

$$(\pi_k, V_k) \to \text{MCTS} \to (\pi'_k, z) \to \text{entrenamiento} \to (\pi_{k+1}, V_{k+1}).$$

17. Relación con Bellman

La ecuación óptima de Bellman para un problema de decisión es:

$$V^*(s) = \max_a \left[r(s,a) + \gamma V^*(T(s,a)) \right].$$

En un juego determinista de suma cero, la alternancia de jugadores introduce un cambio de signo. Una forma simplificada puede escribirse:

$$V^*(s) = \max_a \left[-V^*(T(s,a)) \right]$$

para estados no terminales sin recompensa inmediata.

AlphaZero no aplica una tabla Bellman explícita sobre cada estado. En cambio:

- la red aproxima el valor;
- MCTS realiza una forma de respaldo de valores;
- el resultado terminal proporciona la señal de entrenamiento.

Por ello existe una conexión conceptual profunda con Bellman, pero el mecanismo computacional es distinto de una tabla Q clásica.

18. Valor de una acción

Aunque la red produce principalmente $V(s)$, MCTS mantiene:

$$Q(s,a).$$

Una interpretación es:

$$Q(s,a) \approx -V(T(s,a))$$

desde la perspectiva del jugador que mueve en s, aunque en la práctica Q se obtiene mediante promedios de simulaciones y evaluaciones descendientes.

19. Cómo se define una jugada "mejor"

En una posición concreta, AlphaZero no utiliza un único criterio.
La decisión surge de una combinación de:

$$P(s,a), \quad Q(s,a), \quad N(s,a), \quad U(s,a).$$

Durante la búsqueda:

$$a_{\text{select}} = \arg\max_a \left[Q(s,a) + U(s,a) \right].$$

Al finalizar la búsqueda, la elección suele basarse en:

$$N(s,a).$$

Por tanto, la jugada final no es necesariamente:

$$\arg\max_a P(s,a),$$

ni tampoco:

$$\arg\max_a Q(s,a).$$

Es el resultado acumulado del proceso de búsqueda.

20. Política de entrenamiento versus política de juego

Durante entrenamiento:

$$a_t \sim \pi_t.$$

Esto genera exploración.
Durante evaluación competitiva puede utilizarse:

$$a_t = \arg\max_a N(s_t,a).$$

Esto elimina gran parte de la aleatoriedad.
Esta distinción es esencial:

$$\text{explorar para aprender} \neq \text{explotar para competir}.$$

21. Aprendizaje desde cero

Una propiedad distintiva del enfoque AlphaZero es que no requiere, en principio:

- aperturas humanas;
- bases de finales humanas;
- heurísticas posicionales diseñadas manualmente;
- partidas de grandes maestros.

Su conocimiento emerge de:

$$\text{reglas} + \text{autojuego} + \text{búsqueda} + \text{optimización}.$$

Esto no significa que "no haya conocimiento incorporado". La arquitectura, representación, reglas, parámetros de búsqueda y procedimiento de entrenamiento constituyen fuertes sesgos inductivos.

22. Bootstrap y problema inicial

Al comienzo:

$$p_\theta(a \mid s) \approx \text{ruido estructurado},$$

$$v_\theta(s) \approx 0.$$

Sin embargo, el sistema puede comenzar a aprender porque:

1. juega partidas;
2. obtiene resultados terminales reales;
3. utiliza esos resultados como objetivos de valor;
4. utiliza MCTS para producir objetivos de política;
5. las mejoras pequeñas se retroalimentan.

Este proceso es un *bootstrap*.

23. Replay buffer

Los ejemplos generados se almacenan normalmente en un buffer:

$$\mathcal{B} = \{(s, \pi, z)\}.$$

El entrenamiento extrae mini-batches:

$$B \subset \mathcal{B}.$$

Esto desacopla parcialmente:

$$\text{generación} \quad \text{y} \quad \text{aprendizaje}.$$

Un riesgo importante es entrenar demasiado sobre experiencias obsoletas. Si la política cambia rápidamente, un buffer excesivamente viejo puede contener objetivos π producidos por redes mucho más débiles.

24. Distribución cambiante de datos

AlphaZero aprende de una distribución que él mismo genera:

$$s \sim d_{\pi_\theta}.$$

Cuando cambia la política:

$$\pi_\theta \to \pi_{\theta'},$$

también cambia la distribución:

$$d_{\pi_\theta} \to d_{\pi_{\theta'}}.$$

El problema de aprendizaje es entonces no estacionario.

Esto es una de las razones por las cuales la estabilidad del pipeline de generación y entrenamiento es importante.

25. La interacción búsqueda–red

La relación fundamental puede escribirse como:

$$\text{red} \to \text{búsqueda} \to \text{objetivo} \to \text{red}.$$

Más explícitamente:

$$f_{\theta_k} \to \text{MCTS} \to \{(s, \pi, z)\} \to \theta_{k+1}.$$

La búsqueda actúa como un operador de mejora:

$$\mathcal{I}(f_\theta).$$

El entrenamiento actúa como un operador de compresión:

$$\mathcal{C}(\pi, z) \to f_{\theta'}.$$

Puede verse a la red como una forma de comprimir en parámetros generales el razonamiento realizado por muchas búsquedas anteriores.

26. Por qué el valor terminal puede afectar decisiones lejanas

Supóngase una trayectoria:

$$s_0 \to s_1 \to s_2 \to \cdots \to s_T.$$

Con resultado final:

$$z = -1.$$

El mismo resultado se utiliza como objetivo para múltiples posiciones:

$$(s_0, \pi_0, -1),$$

$$(s_1, \pi_1, +1),$$

$$(s_2, \pi_2, -1),$$

etc., según la perspectiva del jugador al turno.

Esto permite que una consecuencia negativa lejana modifique el valor aprendido de posiciones anteriores.

Sin embargo, esa propagación no es exacta ni instantánea. Depende de:

- frecuencia con que reaparezcan estados similares;
- capacidad de la red;
- distribución de entrenamiento;
- calidad de MCTS;
- tasa de aprendizaje;
- interferencia con otros ejemplos.

27. Diferencia entre retropropagación MCTS y backpropagation neuronal

Conviene no confundir dos procesos.

27.1. Backup en MCTS

Propaga un valor a través del árbol:

$$v \to Q(s, a).$$

27.2. Backpropagation neuronal

Propaga derivadas:

$$\nabla_\theta \mathcal{L}$$

a través de las capas de la red.

Son mecanismos completamente diferentes.

28. No existe una garantía práctica de optimalidad

En teoría, ciertos algoritmos MCTS tienen propiedades asintóticas bajo supuestos específicos. Pero AlphaZero real utiliza:

- un número finito de simulaciones;
- una red aproximadora;
- búsqueda selectiva;
- entrenamiento estocástico;
- datos generados por políticas cambiantes.

Por tanto, en una posición concreta no puede afirmarse:

$$a_{\text{AlphaZero}} = a^*.$$

Lo correcto es afirmar que AlphaZero intenta aproximar una política fuerte mediante mejora iterativa.

29. Error de aproximación

Podemos descomponer conceptualmente el error total en varias fuentes:

$$\epsilon_{\text{total}} \approx \epsilon_{\text{red}} + \epsilon_{\text{search}} + \epsilon_{\text{data}} + \epsilon_{\text{optimization}} + \epsilon_{\text{sampling}}.$$

Donde:

- ϵ_{red}: limitación de representación;
- ϵ_{search}: búsqueda insuficiente;
- ϵ_{data}: cobertura incompleta;
- $\epsilon_{\text{optimization}}$: entrenamiento imperfecto;
- $\epsilon_{\text{sampling}}$: ruido estadístico.

30. Presupuesto de búsqueda

Sea:

$$M$$

el número de simulaciones por jugada.
En general, aumentar:

$$M$$

mejora la calidad de la búsqueda, pero incrementa el costo:

$$C_{\text{move}} \propto M \cdot C_{f_\theta}.$$

No obstante, la mejora no es lineal. Normalmente existe rendimiento decreciente.

31. Uso de batching

Las evaluaciones neuronales pueden agruparse:

$$\{s_1, \ldots, s_B\} \to f_\theta.$$

Esto aprovecha hardware paralelo.
El problema es que MCTS es secuencial en parte:

$$\text{selección} \to \text{evaluación} \to \text{backup}.$$

La implementación eficiente requiere coordinar muchas búsquedas simultáneas.

32. Virtual loss

Cuando varias simulaciones se ejecutan en paralelo, pueden seleccionar la misma rama. Una técnica frecuente es introducir temporalmente una penalización:

$$Q_{\text{temp}}(s, a) = Q(s, a) - L_{\text{virtual}}.$$

Esto reduce colisiones entre hilos.

33. Entrenamiento síncrono y asíncrono

Un sistema distribuido puede contener:

self-play workers,

learner,

parameter server.

Los workers generan partidas con una versión:

$$\theta_k.$$

El learner produce:

$$\theta_{k+1}.$$

Un riesgo sistémico es el *policy lag*: partidas nuevas pueden estar siendo generadas todavía con una política vieja.

Si la tasa de generación supera ampliamente la tasa de entrenamiento:

$$R_{\text{selfplay}} \gg R_{\text{learn}},$$

el buffer acumula experiencias obsoletas.

Este fenómeno es especialmente relevante en implementaciones prácticas.

34. Distribución de visitas como mejora de política

Puede interpretarse que MCTS implementa un operador:

$$\mathcal{M}_M : (p_\theta, v_\theta) \mapsto \pi_M.$$

Cuando M aumenta, la política de visitas incorpora más información de búsqueda.

El entrenamiento intenta resolver:

$$p_{\theta'} \approx \pi_M.$$

Así, la red intenta absorber parte del conocimiento computado por la búsqueda.

35. Interpretación como búsqueda amortizada

Supóngase que una búsqueda costosa encuentra repetidamente patrones de buenas decisiones.

El entrenamiento intenta codificarlos en:

$$\theta.$$

Esto puede interpretarse como *amortized search*:

búsquedas costosas pasadas $\rightarrow$ predicciones baratas futuras.

La red funciona como memoria generalizada de muchos procesos de búsqueda.

36. Relación entre política y valor

Las dos cabezas están relacionadas pero no son redundantes.
La política responde:

$$\text{£dónde buscar?}$$

El valor responde:

$$\text{£qué tan buena es la posición?}$$

PUCT utiliza:

$$P$$

para priorizar expansión y

$$Q$$

para incorporar resultados acumulados.
Una política excelente con un valor pobre puede orientar bien inicialmente pero evaluar mal las hojas.
Un valor excelente con una política pobre puede evaluar correctamente posiciones pero desperdiciar búsquedas en ramas irrelevantes.

37. Aperturas

AlphaZero no necesita un libro de aperturas explícito.
Si determinadas secuencias producen buenos resultados, las probabilidades:

$$p_\theta(a \mid s)$$

y las visitas:

$$N(s, a)$$

tienden a concentrarse en ellas.
El "libro" queda implícitamente distribuido entre:

$$\theta$$

y el comportamiento de MCTS.

38. Finales

En finales, la calidad depende de:

- cobertura del autojuego;
- capacidad de la red;
- profundidad efectiva de MCTS.

AlphaZero no posee necesariamente una tabla de finales exacta.
Por tanto, puede aproximar muy bien finales frecuentes sin resolver matemáticamente todos ellos.

39. Estados transpuestos

En juegos como ajedrez, diferentes secuencias pueden llegar a la misma posición:

$$s_i = s_j.$$

Una implementación puede aprovechar:

transposition tables

o caches de evaluación.

El AlphaZero conceptual se describe normalmente como árbol, aunque implementaciones prácticas pueden compartir información entre estados idénticos.

40. Simetrías

En juegos con simetrías, un estado puede transformarse:

$$s \to g(s)$$

sin alterar la estructura estratégica esencial.

Puede utilizarse augmentación:

$$(s, \pi, z) \to (g(s), g(\pi), z).$$

En ajedrez las simetrías útiles son más restringidas que en Go debido a características como color, orientación de peones y reglas especiales.

41. Calibración del valor

Idealmente:

$$v_\theta(s) \approx \mathbb{E}[z \mid s].$$

Pero una red puede estar bien ordenada sin estar perfectamente calibrada.

Por ejemplo:

$$v(s_1) > v(s_2)$$

puede ser correcto aunque sus magnitudes no representen probabilidades exactas.

La fuerza de juego depende tanto del orden relativo como de la calibración.

42. Política multimodal

Si varias jugadas son equivalentes:

$$Q(s, a_1) \approx Q(s, a_2),$$

MCTS puede distribuir visitas entre ellas.

La política objetivo puede ser:

$$\pi(a_1 \mid s) = 0.45, \qquad \pi(a_2 \mid s) = 0.50,$$

en lugar de imponer una única jugada.

Esto proporciona una señal de aprendizaje más rica que una etiqueta discreta.

43. Crédito temporal

Una dificultad fundamental del aprendizaje por refuerzo es asignar crédito a decisiones antiguas.

AlphaZero lo aborda indirectamente mediante:

1. resultado terminal z;
2. entrenamiento del valor en cada estado;
3. búsqueda que explora consecuencias;
4. repetición de patrones a lo largo de muchas partidas.

No existe una garantía de que una mala decisión aislada sea identificada perfectamente después de una sola partida.

44. Una formulación compacta del algoritmo

Inicialización

Inicializar:

$$\theta_0.$$

Iteración k

Para múltiples partidas:

$$s_0 \leftarrow \text{posición inicial.}$$

Mientras la partida no termine:

$$(\boldsymbol{p}, v) = f_{\theta_k}(s_t),$$

$$\pi_t = \mathrm{MCTS}(s_t, f_{\theta_k}),$$

$$a_t \sim \pi_t,$$

$$s_{t+1} = T(s_t, a_t).$$

Cuando termina:

$$z = \text{resultado}\,.$$

Guardar:

$$(s_t, \pi_t, z_t).$$

Después actualizar:

$$\theta_{k+1} = \arg\min_{\theta} \mathbb{E}_{(s,\pi,z)\sim\mathcal{B}} \left[(z - v_\theta(s))^2 - \pi^\top \log p_\theta(s) + \lambda \|\theta\|^2 \right].$$

45. Pseudocódigo conceptual

```
initialize neural network f_theta
initialize replay buffer B

repeat:
    for each self-play game:
        s = initial_state
        trajectory = []

        while s is not terminal:
            pi = MCTS(s, f_theta)
            trajectory.append((s, pi))
            a ~ pi
            s = next_state(s, a)

        z = game_result(s)

        for (s_t, pi_t) in trajectory:
            B.add(s_t, pi_t, result_from_player_perspective(z, s_t))

    sample minibatches from B

    update theta minimizing:
        (z - v_theta(s))^2
        - pi^T log p_theta(s)
        + lambda ||theta||^2
```

46. Qué significa que AlphaZero “aprende estrategia”

No existe una variable interna llamada “estrategia”.
El conocimiento estratégico queda distribuido en los parámetros:

$$\theta.$$

Puede manifestarse en:

- valoración de estructura;
- preferencia por actividad;
- seguridad del rey;
- valoración de espacio;
- sacrificios posicionales;
- coordinación de piezas;
- transición favorable a finales.

Estos conceptos pueden emerger sin estar etiquetados explícitamente.

47. Conocimiento implícito

Una representación interna puede contener una característica latente:

$$h_j(s)$$

que correlacione con un concepto humano como seguridad del rey.
Pero la red no necesita nombrar ese concepto.
El conocimiento es distribuido:

$$\theta = \{\theta_1, \theta_2, \ldots, \theta_n\}.$$

Esto dificulta la auditabilidad.

48. Auditabilidad

Una debilidad práctica de AlphaZero es que la explicación causal de una decisión no es inmediata.
Puede observarse:

$$P(s,a), \quad Q(s,a), \quad N(s,a),$$

pero no siempre puede reconstruirse un razonamiento humano equivalente.
Para auditar aprendizaje conviene registrar:

- política previa;
- visitas por acción;
- valores Q;
- valor de red;
- versión del modelo;
- profundidad efectiva;
- número de nodos;
- resultado posterior.

49. Diagnóstico de aprendizaje

Un sistema puede parecer estar aprendiendo porque la pérdida disminuye:

$$\mathcal{L} \downarrow$$

sin mejorar realmente su fuerza.
Por ello deben medirse por separado:

1. precisión de política;
2. error de valor;
3. calibración;
4. fuerza contra referencias estables;
5. estabilidad entre versiones;
6. diversidad del autojuego.

50. Sobreajuste al autojuego

Si el sistema juega siempre contra sí mismo, puede explotar debilidades particulares de su propia distribución.

Formalmente aprende sobre:

$$d_{\pi_\theta}.$$

Pero una política externa induce:

$$d_{\pi_{\text{opp}}}.$$

Si ambas distribuciones difieren mucho, puede aparecer un problema de generalización.

51. Evaluación externa

Es conveniente utilizar oponentes de referencia:

$$\mathcal{O}_1, \mathcal{O}_2, \ldots$$

y posiciones fijas:

$$\mathcal{S}_{\text{test}}.$$

Así puede medirse si una nueva versión mejora de forma general o sólo se adapta a su propio estilo.

52. Catastrophic forgetting

El entrenamiento continuo puede mejorar unas regiones del espacio de estados y empeorar otras.

Esto se denomina *catastrophic forgetting*.

Un replay buffer suficientemente diverso ayuda a mitigarlo, pero introduce el problema opuesto de obsolescencia.

Por tanto existe una tensión:

$$\text{recencia} \quad \text{vs} \quad \text{diversidad histórica}.$$

53. Parámetros esenciales

Entre los parámetros más importantes se encuentran:

Parámetro	**Función**
M	Número de simulaciones MCTS por jugada.
c_{puct}	Balance entre exploración y explotación.
τ	Temperatura de la política de visitas.
ε	Peso del ruido Dirichlet.
α	Concentración del ruido Dirichlet.
η	Tasa de aprendizaje.
λ	Regularización L_2.
B	Tamaño de mini-batch.
$\lvert\mathcal{B}\rvert$	Tamaño del replay buffer.

54. Efecto del número de simulaciones

Si el prior de red es incorrecto pero existe suficiente búsqueda, MCTS puede corregirlo.
Con pocas simulaciones:

$$\pi \approx p_\theta.$$

Con más simulaciones:

$$\pi$$

puede desviarse sustancialmente de:

$$p_\theta.$$

Por eso el número de simulaciones controla parcialmente cuánto "razonamiento adicional" se realiza sobre la red.

55. Profundidad versus anchura

AlphaZero no utiliza una profundidad fija tradicional.
MCTS distribuye recursos de forma adaptativa.
Una variante prometedora puede recibir:

$$N_1 \gg N_2.$$

Así, la profundidad efectiva depende del interés de la rama.
Esto es conceptualmente diferente de:

$$\text{minimax depth-}d.$$

56. Comparación con minimax

Minimax clásico busca:

$$V_d(s) = \max_a \min_b \cdots$$

hasta profundidad fija d, utilizando una función heurística en las hojas.
AlphaZero reemplaza:

- profundidad fija por expansión selectiva;
- heurística manual por red neuronal;
- ordenamiento manual por política aprendida.

57. Comparación con Q-learning

Q-learning intenta aproximar:

$$Q^*(s, a).$$

AlphaZero modela principalmente:

$$p(a \mid s), \qquad v(s).$$

MCTS genera estimaciones:

$$Q_{\text{MCTS}}(s, a).$$

Una ventaja es separar:

$$\text{prior de acción}$$

de:

$$\text{evaluación del estado.}$$

58. Comparación con policy gradient puro

Un policy gradient puro actualiza directamente:

$$\nabla_\theta \mathbb{E}[z].$$

AlphaZero utiliza una política objetivo más informativa:

$$\pi_{\text{MCTS}}.$$

Por ello cada estado contiene una señal estructurada sobre múltiples acciones, no solamente el resultado terminal.

59. El papel de MCTS como "teacher"

Puede interpretarse MCTS como un maestro generado dinámicamente.
La red actual:

$$f_{\theta_k}$$

crea una búsqueda que produce:

$$\pi_k.$$

Luego esa política se convierte en objetivo para:

$$f_{\theta_{k+1}}.$$

Es una forma de auto-destilación guiada por búsqueda.

60. Riesgo de auto-confirmación

Existe un riesgo conceptual:

$$\text{red sesgada} \rightarrow \text{búsqueda sesgada} \rightarrow \text{datos sesgados.}$$

La exploración, ruido, autojuego diverso y suficiente presupuesto de búsqueda son mecanismos que ayudan a romper ese circuito.

61. Convergencia: qué puede y qué no puede afirmarse

En un sistema ideal tabular, ciertos esquemas de policy iteration poseen garantías fuertes. AlphaZero viola muchas hipótesis simples:

- aproximación funcional;
- entrenamiento estocástico;
- búsqueda limitada;
- distribución cambiante;
- optimización no convexa.

Por tanto, no existe una garantía general práctica de:

$$\theta_k \to \theta^*$$

ni de:

$$\pi_{\theta_k} \to \pi^*.$$

El éxito se establece empíricamente.

62. Una lectura sistémica

AlphaZero puede verse como un sistema de tres niveles:

$$\boxed{\text{red} \to \text{búsqueda} \to \text{aprendizaje}}$$

La red aporta velocidad.
La búsqueda aporta deliberación.
El aprendizaje convierte deliberación pasada en conocimiento reutilizable.
La calidad total depende del equilibrio entre los tres.

63. Implicancias para implementaciones experimentales

Para un motor experimental que busque inspiración en AlphaZero, conviene preservar conceptualmente las siguientes separaciones:

1. **evaluación de posición**;
2. **prior de jugadas**;
3. **búsqueda**;
4. **generación de experiencia**;
5. **aprendizaje**;
6. **persistencia**;
7. **evaluación independiente**.

Mezclar estos niveles dificulta diagnosticar si una mejora proviene de:

$$\text{mejor red}, \quad \text{más búsqueda}, \quad \text{mejores datos},$$

o simplemente:

$$\text{más costo computacional}.$$

64. Qué debería medirse

Una implementación auditable debería registrar al menos:

$$\text{policy entropy},$$

$$\text{value error},$$

$$\text{search improvement},$$

$$D_{\mathrm{KL}}(\pi_{\mathrm{MCTS}} \,\|\, p_\theta),$$

$$\text{win rate},$$

$$\text{draw rate},$$

$$\text{search nodes},$$

$$\text{latency},$$

$$\text{model version}.$$

Una métrica particularmente útil es:

$$\Delta_{\text{search}} = V_{\text{after search}} - V_{\text{network}}.$$

Aunque su definición concreta requiere cuidado, conceptualmente mide cuánto valor adicional aporta la búsqueda.

65. Desacople entre generación y aprendizaje

Si se utilizan muchos workers, la arquitectura puede representarse como:

$$W_1, W_2, \ldots, W_n \to Q \to L.$$

Donde:

- W_i: workers de autojuego;
- Q: cola o replay buffer;
- L: learner.

La condición deseable no es simplemente maximizar throughput.
Debe controlarse la edad de experiencia:

$$A = k_{\text{current}} - k_{\text{generator}}.$$

Un valor alto de A indica que se está entrenando o generando con políticas antiguas.

66. Edad de política y coherencia

Si una partida comenzó con:

$$\theta_k,$$

es conceptualmente limpio que termine con la misma versión o con una política coherente.
Cambiar la red a mitad de una partida produce una trayectoria generada por:

$$\theta_k, \theta_{k+1}, \ldots$$

lo que dificulta interpretar los datos.
No es necesariamente inválido, pero reduce la trazabilidad experimental.

67. Conclusiones

AlphaZero no es simplemente una red neuronal que elige jugadas, ni un MCTS clásico, ni un algoritmo Bellman tabular. Su fuerza proviene de un circuito cerrado:

$$\text{predicción} \to \text{búsqueda} \to \text{experiencia} \to \text{aprendizaje}.$$

La red proporciona:

$$(p, v).$$

PUCT utiliza estas estimaciones para construir una búsqueda selectiva.
La búsqueda produce:

$$\pi.$$

El resultado terminal produce:

$$z.$$

El entrenamiento ajusta:

$$p_\theta \to \pi, \qquad v_\theta \to z.$$

El proceso se repite.

La "mejor estrategia" no aparece como una fórmula cerrada ni como una tabla exacta. Emerge progresivamente de la interacción entre autojuego, búsqueda y aproximación neuronal.

La conclusión técnicamente más importante es que AlphaZero no elimina la búsqueda: aprende a hacerla más eficiente. Tampoco elimina la necesidad de evaluación: aprende una función de valor. Y tampoco demuestra optimalidad: construye una aproximación cada vez más fuerte dentro de un presupuesto computacional finito.

68. Resumen matemático final

La red:

$$f_\theta(s) = (p_\theta(\cdot \mid s), v_\theta(s)).$$

Selección PUCT:

$$a = \arg\max_a \left[Q(s,a) + c_{\text{puct}} P(s,a) \frac{\sqrt{\sum_b N(s,b)}}{1 + N(s,a)} \right].$$

Política MCTS:

$$\pi(a \mid s) = \frac{N(s,a)^{1/\tau}}{\sum_b N(s,b)^{1/\tau}}.$$

Ejemplo de entrenamiento:

$$(s, \pi, z).$$

Pérdida:

$$\mathcal{L}(\theta) = (z - v_\theta(s))^2 - \sum_a \pi(a \mid s) \log p_\theta(a \mid s) + \lambda \|\theta\|_2^2.$$

Actualización:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}.$$

Ciclo:

$$f_{\theta_k} \to \text{MCTS} \to (s, \pi, z) \to f_{\theta_{k+1}}.$$

A. Glosario

Estado s configuración completa del juego.

Acción a jugada legal.

Política p distribución de jugadas producida por la red.

Política π distribución mejorada producida por MCTS.

Valor v predicción de resultado.

Prior P probabilidad inicial asignada a una acción en MCTS.

Visitas N número de veces que una arista fue explorada.

Valor medio Q evaluación acumulada de una acción.

PUCT regla que combina valor y exploración guiada por prior.

Replay buffer memoria de experiencias de entrenamiento.

Self-play autojuego.

Policy lag retraso entre la política actual y la que generó los datos.

Bootstrap mejora progresiva usando las propias estimaciones.

B. Referencias conceptuales sugeridas

Para profundizar en el tema conviene consultar los trabajos originales de AlphaGo Zero y AlphaZero, además de literatura sobre Monte Carlo Tree Search, aprendizaje por refuerzo, policy iteration, redes residuales y métodos actor–critic.